Project Specification: Lights Out

2.1 Project Introduction

• **Team members**: Ricky Raveanu (RickyRAV), Nikola (Nikola-Petro)

• **Working title**: Lights Out

• **Summary**: A multiplayer Formula 1 prediction game. Before each race weekend's session lockout, everyone in a friend group privately predicts the finishing order. Once the real session ends, the app auto-grades everyone against the actual result and updates a shared season leaderboard.

• **Target users / problem**: F1 fans who already trade predictions informally in group chats or casual bets, but have no real way to track who's actually right over a season. Lights Out turns that into an actual record: predictions are sealed so nobody can copy, results are graded automatically, and the leaderboard keeps score all season.

2.2 Chosen Framework

• **Framework**: React Native via Expo (managed workflow).

• **Rationale**: both of us already know React and TypeScript, so most of the learning curve is the mobile-specific stuff (native APIs, navigation, platform quirks), not the language itself.

• **Target platforms**: Android is developed and tested by both.

2.3 Core User Stories

1. **As a fan, I want to see the upcoming race with a countdown to lockout, so I know how long I have to predict.**

```
GIVEN the app is opened,
WHEN the Predict screen loads,
THEN the next race's name, date, and a live countdown are shown, sourced from our own API.
Done when the countdown reflects the real current-season schedule.
```

2. **As a fan, I want to submit a sealed top-5 prediction before lockout, so I can compete without anyone copying my picks.**

```
GIVEN the prediction window is open,
WHEN I order 5 drivers and submit,
THEN it's saved to the shared backend, becomes uneditable after lockout,
AND stays hidden from friends until lockout passes (only a submitted/not-submitted flag
  is visible to them).
Done when a submitted prediction survives an app restart and is never returned to another
  user pre-lockout.
```

3. **As a fan, I want to see which friends have already submitted, so I know who's still deciding.**

```
GIVEN the prediction window is open,
WHEN I view the race screen,
THEN each friend's submitted/not-yet status is shown without revealing picks.
Done when the status list updates as friends submit.
```

4. **As a fan, I want a push notification before lockout, so I don't miss my chance to predict.**

```
GIVEN notification permission is granted and I haven't predicted yet,
WHEN lockout enters a configured reminder window,
THEN I get a local notification.
Done when it fires on both Android and iOS, and permission-denied is handled without
  breaking the app.
```

5. **As a fan, I want my prediction automatically scored against the real result, so I don't calculate points myself.**

```
GIVEN a race has finished,
WHEN the backend fetches the final classification from Jolpica-F1,
THEN every prediction for that race is scored and stored.
Done when the scoring function is unit-tested against known predicted/actual pairs.
```

6. **As a fan, I want a shared season leaderboard for my friend group, so I can see who's winning overall.**

```
GIVEN at least one race is scored,
WHEN I open the Leaderboard screen,
THEN members are ranked by total season points from the shared backend.
Done when a newly graded race is reflected without any manual step.
```

7. **As a fan, I want to browse the calendar, my history, and the last-synced leaderboard while offline, so the app stays useful without a connection.**

```
GIVEN I've previously loaded the app online,
WHEN I open it offline,
THEN cached data is still viewable with a "last updated" indicator,
AND attempting to submit a prediction offline shows a clear "reconnect to submit" message
  instead of losing my draft or crashing.
Done when none of these screens ever show a blank/broken state offline.
```

8. **As a fan, I want to create my friend group or join it with an invite code right after registering, so my predictions count toward a leaderboard with people I know.**

```
GIVEN I've just registered and I'm not in a group yet,
WHEN I create a group or enter a valid invite code,
THEN I'm in that group for good and appear on its leaderboard going forward.
Done when an invalid code is rejected without creating a membership, and a user who's
  already in a group can't create or join another one.
```

2.4 Screen / Navigation Structure

• **Login / Register** (auth stack): email/password sign-in or account creation. Successful authentication opens the main tabs, or onboarding first if the user isn't in a group yet. An existing valid session skips Login. Signing out from Settings returns to Login.

• **Onboarding** (once, right after registering): create a friend group and share its invite code, or join one with a code. Every user is in exactly one group, and there's no leaving or switching, so a returning user goes straight to the tabs.

• **Predict** (tab, stack root): next race, countdown, prediction form or sealed "submitted" state → **Race Detail** (stack): full session info, friend status list, and (post-lockout) everyone's revealed picks/points.

• **Leaderboard** (tab): season standings.

• **History** (tab): past predictions, results, points earned.

• **Settings** (tab): notifications, your friend group's name and invite code, account.

Planned navigation flow:

```
App launch -> Session check
  No valid session -> Login <-> Register -> Group check (after authentication)
  Valid session ------------------------> Group check
Group check:
  No group yet -> Onboarding (create or join) -> Main tabs
  Already in a group -------------------------> Main tabs
Main tabs:
  Predict -> Race Detail (stack, back returns to Predict)
  Leaderboard
  History
  Settings -> Sign out -> Login
```

Key state: auth session, the user's friend group, cached calendar/results (TanStack Query), in-progress prediction draft (Zustand), notification permission status.

2.5 API and Data Sources

1. **Jolpica-F1** (`api.jolpi.ca/ergast/f1/`): a free REST API, no auth needed, compatible with the old Ergast API. Only our backend calls it, never the app directly. That way scoring can't be gamed client-side, and we're not stuck if Jolpica-F1 ever changes shape.
2. **Self-hosted backend** (running on Ricky's own server): the app's only backend. It holds predictions, friend groups, and the leaderboard, and proxies and caches the race calendar and final results from Jolpica-F1 so the app never talks to a third party directly. The app reads and writes to it through TanStack Query. After a race ends, the backend fetches the result from Jolpica-F1 itself and grades everyone.

Every screen talks only to our own backend. For auth we're using a simple self-issued JWT with email/password, nothing more since it's just our friend group and not a public app.

2.6 Native Feature Usage

Local push notifications (`expo-notifications`) remind a user to predict before lockout.

2.7 Offline / Persistence Strategy

- **Read, fully offline**: the calendar, history, and last-synced leaderboard are cached (TanStack Query's persisted cache), so they're browsable with no internet. We show a "last updated" timestamp so it's clear when it's stale.
- **Write, needs connectivity**: a prediction only counts once it reaches the backend, so submitting one can't really happen offline. If you try without a connection, we keep your draft locally and tell you to reconnect before lockout instead of just crashing or dropping it silently.

We talked about building a real offline write-queue (save predictions locally, sync once you're back online), but decided to leave that for later if there's time. Keeping it out of the baseline is more realistic for one semester.

2.8 Technical Approach & Architecture

The planned mobile architecture is layered: presentation (`screens/`, `components/`), API access (`hooks/`), and local/UI state (`store/`). Screens compose these layers, keeping API access and state management outside the view code. Navigation uses an authentication stack and a one-time onboarding step, followed by bottom tabs, with a nested stack for Predict and Race Detail. The backend uses a three-layer architecture (routes/API, business logic, data access).

One GitHub repo (Moodle takes a single URL), pnpm workspaces:

```
apps/mobile/   Expo app (screens/, components/, store/, hooks/)
apps/api/      Fastify + Drizzle backend (auth, groups, predictions, scoring, leaderboard)
docker-compose.yml        local dev (db + api)
docker-compose.prod.yml   deployed on Ricky's own server
```

Mobile's `store/` folder is Zustand, for local/UI state. The `hooks/` folder wraps TanStack Query calls to our own API. Styling is done with NativeWind (Tailwind for React Native), which gives us full control over a custom dark racing theme instead of a default Material look. For components we don't want to build from scratch (forms, modals, lists), we're pulling in gluestack-ui's primitives, which copy the component code straight into our repo and get restyled with our own theme, rather than pulling in a locked dependency.

The backend is built with Fastify and Drizzle on top of PostgreSQL, split into three layers: routes/API, business logic, and data access, so each part has one clear job. Incoming request bodies are validated with Zod before they reach the business logic layer, so bad input gets rejected early with a clear error instead of failing deeper in the code. It's deployed as a container on Ricky's own server, using the same docker-compose setup for local development and production, so both of us develop against one shared, always-on backend instead of two separate local databases. Screens don't call `fetch` directly, they always go through a query or mutation hook. For testing, we're covering the scoring algorithm with unit tests and doing at least one component test on the leaderboard/prediction card.